

[ACT] S - 11

Manejo de Multisesiones y Persistencia

Alumno

Oscar Espinoza Landeta

Docente

José Miguel Carrera Pacheco

Fecha de entrega: 16 de marzo de 2026

1. ¿Qué se investigó?

1.1 ¿Qué es una multisesión?

Una multisesión ocurre cuando un mismo usuario mantiene varias sesiones activas de forma simultánea. Ejemplos cotidianos: la misma cuenta abierta en Chrome y Firefox al mismo tiempo, el mismo usuario autenticado en su celular y en su laptop, o múltiples pestañas del mismo sistema abiertas en un navegador.

En sistemas con JWT stateless (sin estado), el servidor no "recuerda" qué tokens emitió. Cada token es autosuficiente y válido hasta su fecha de expiración, sin importar cuántos existan. Esto hace que la multisesión sea el comportamiento por defecto, pero sin control alguno.

1.2 Problemas con sesiones concurrentes

- **Inconsistencia de estado:** Si dos pestañas modifican datos simultáneamente pueden sobrescribirse mutuamente.
- **Logout incompleto:** Si el usuario cierra sesión en una pestaña, las otras siguen activas con el mismo token.
- **Tokens huérfanos:** Tokens emitidos que nunca fueron revocados siguen siendo válidos aunque el usuario ya no los use.
- **Robo de sesión:** Un token robado sigue siendo válido hasta su expiración natural (horas o días).
- **Desincronización de UI:** Una pestaña muestra datos desactualizados porque otra pestaña ya modificó el estado del servidor.

1.3 ¿Cómo se invalida una sesión?

Con JWT puro es imposible invalidar un token antes de su expiración porque el servidor no guarda estado. La solución es un enfoque híbrido:

1. Se añade un identificador único (jti — JWT ID) al payload del token.
2. Ese jti se persiste en la base de datos junto con metadatos de la sesión (dispositivo, IP, expiración).
3. En cada petición, el middleware verifica la firma JWT (criptografía) Y consulta la BD para confirmar que la sesión sigue activa.
4. Para invalidar: se marca `is_active = FALSE` en la BD. El token sigue siendo criptográficamente válido, pero el servidor lo rechaza.

1.4 Riesgos si no se controlan

- Escalamiento de privilegios: un token con rol elevado robado se puede usar hasta que expire.
- Sesiones fantasma: usuarios dados de baja que siguen autenticados en el sistema.
- Agotamiento de recursos: sesiones acumuladas sin limpiar pueden llenar la base de datos.
- Violaciones de cumplimiento: regulaciones como GDPR exigen poder revocar acceso inmediatamente.

2. ¿Para qué se implementó?

El objetivo fue que el sistema Tag Human Universal funcione correctamente en escenarios reales de uso, considerando que los usuarios:

- Abren varias pestañas del sistema al mismo tiempo.
- Inician sesión desde distintos dispositivos (laptop, celular, computadora del trabajo).
- Pierden conexión temporalmente y esperan recuperar su sesión al reconectarse.
- Cierran sesión en un dispositivo y esperan que todos los demás también sean cerrados.

Con esto se garantiza:

- **Multisessiones funcionales:** Un usuario puede tener activas sesiones desde múltiples dispositivos sin interferencia entre ellas.
- **Recuperación de sesión válida:** Al recargar la página, el sistema valida con el servidor si el token sigue vigente antes de restaurar el estado autenticado.
- **Control de expiración:** Las sesiones expiran automáticamente (JWT nativo a las 8 horas) y un job de limpieza cada 30 minutos actualiza la BD.
- **Evitar inconsistencias de estado:** Mediante BroadcastChannel, si una pestaña cierra sesión, todas las demás pestañas abiertas del mismo navegador lo reflejan inmediatamente.

3. ¿Qué se hizo?

3.1 Arquitectura de la solución

Se implementó un sistema de sesiones persistentes basado en la combinación de JWT + tabla de sesiones en PostgreSQL. El núcleo del enfoque es el campo jti (JWT ID): un UUID único que se incluye en cada token y se persiste en la BD para permitir su revocación individual.

3.2 Base de datos — Tabla sessions

Se creó la migración backend/db/sessions_migration.sql con la siguiente estructura:

```
CREATE TABLE sessions (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  jti         UUID NOT NULL UNIQUE,          -- JWT ID único por token  
  device_info TEXT,                          -- User-Agent del  
dispositivo  
  ip_address  INET,                          -- IP del cliente  
  created_at  TIMESTAMP DEFAULT NOW(),  
  last_activity_at TIMESTAMP DEFAULT NOW(),  -- Heartbeat  
  expires_at  TIMESTAMP NOT NULL,           -- Igual que expiración del  
JWT  
  is_active   BOOLEAN DEFAULT TRUE           -- FALSE = sesión revocada  
);
```

Decisiones de diseño:

- jti UUID UNIQUE: garantiza que cada token es identificable e irrepetible.
- ON DELETE CASCADE: si el usuario es eliminado, sus sesiones desaparecen automáticamente.
- is_active separado de expires_at: permite revocar manualmente sin esperar que el JWT expire.
- Índices en user_id, jti, is_active y expires_at para consultas eficientes.

3.3 Backend — Archivos creados / modificados

- **authController.js (modificado)**: Login y registro ahora generan un jti con crypto.randomUUID() y llaman a createSession() para registrar la sesión en BD. Cada login crea una sesión nueva sin borrar las anteriores (multisesión permitida).
- **sessionController.js (nuevo)**: Contiene la lógica de negocio: createSession, validateSession, updateSessionActivity, listSessions, revokeSession, revokeAllSessions, logout, validateCurrentSession.
- **sessionMiddleware.js (nuevo)**: Reemplaza al authMiddleware con doble validación: (1) verifica firma JWT, (2) consulta BD por jti activo. Además actualiza last_activity_at en cada petición.
- **sessionRoutes.js (nuevo)**: Expone 5 endpoints REST: GET /api/sessions, GET /api/sessions/validate, POST /api/sessions/logout, DELETE /api/sessions/all, DELETE /api/sessions/:id.
- **sessionCleanup.js (nuevo)**: Job que se ejecuta cada 30 minutos y marca como is_active = FALSE todas las sesiones cuyo expires_at ya pasó.

- **app.js (modificado):** Registra las nuevas rutas de sesión y arranca el cleanup job al iniciar el servidor.

3.4 Frontend — Archivos creados / modificados

- **sessionManager.js (nuevo):** Utilidades para guardar/limpiar sesión en localStorage, llamadas a la API de sesiones, y BroadcastChannel para emitir/recibir eventos SESSION_STARTED y SESSION_ENDED entre pestañas.
- **useSession.js (nuevo):** Hook React central. Al montar, verifica la sesión con el servidor (recuperación). Escucha eventos BroadcastChannel para sincronizar entre pestañas. Revalida contra el servidor cada 5 minutos. Expone: isAuthenticated, user, isLoading, login(), logout(), logoutAll().
- **App.jsx (modificado):** Integra useSession. Muestra un spinner mientras se verifica la sesión al cargar la página. El handleLogout es ahora asíncrono y llama al servidor antes de limpiar el estado local.

3.5 Endpoints REST de sesión

Método	Ruta	Descripción
GET	/api/sessions	Lista las sesiones activas del usuario autenticado (con flag is_current).
GET	/api/sessions/validate	Verifica si la sesión actual sigue activa en BD.
POST	/api/sessions/logout	Cierra la sesión actual (invalida solo este token).
DELETE	/api/sessions/all	Cierra TODAS las sesiones del usuario (todos los dispositivos).
DELETE	/api/sessions/:id	Revoca una sesión específica por UUID.

3.6 Tests de sesión

Se implementaron 14 tests en backend/tests/session.test.js usando Jest. Los tests cubren 8 escenarios críticos con mocks de BD en memoria para ejecutarse sin necesidad de una base de datos real:

1. Creación de sesión al hacer login

- ✓ Crea sesión en BD con jti único
- ✓ El token JWT contiene el jti como claim

2. Multisesión — múltiples sesiones activas

- ✓ Usuario puede tener N sesiones simultáneas
- ✓ Cada sesión tiene un jti distinto

3. Validación de sesión activa

- ✓ Sesión activa es válida
- ✓ jti inexistente retorna falsy
- ✓ Sesión inactiva es rechazada

4. Logout individual

- ✓ Logout invalida solo la sesión actual, no las demás

5. Logout masivo

- ✓ Revocar todas cierra todos los dispositivos
- ✓ No afecta sesiones de otros usuarios

6. Control de expiración

- ✓ Sesión con expires_at pasado es inválida
- ✓ JWT expirado lanza excepción

7. Sincronización entre pestañas

- ✓ Evento SESSION_ENDED se emite al limpiar sesión

8. Limpieza de sesiones expiradas

- ✓ Cleanup marca inactivas solo las expiradas, no las válidas

Tests: 14 / 14 pasando ✓ — Tiempo de ejecución: 1.158 s

4. ¿Qué se aprendió?

Estado distribuido

Cuando un sistema tiene múltiples clientes (pestañas, dispositivos), el estado de sesión no puede vivir solo en el token. Se necesita una fuente de verdad centralizada (la BD) que pueda ser consultada en tiempo real. Un JWT válido criptográficamente puede no ser una sesión válida desde el punto de vista del negocio.

Pensar en usuarios reales

Los usuarios reales no cierran el navegador correctamente, abren el mismo sistema en varios dispositivos, pierden conexión y esperan que el sistema los "recuerde", y olvidan que dejaron una sesión abierta en un dispositivo compartido. Diseñar para el "camino feliz" no es suficiente; cada uno de estos escenarios requiere un mecanismo específico.

Seguridad vs. Usabilidad

Existe una tensión constante entre seguridad y experiencia de usuario. El balance implementado fue: tokens con jti rastreable y revocación inmediata posible (seguridad), pero en modo offline no expulsamos al usuario y recuperamos la sesión automáticamente al reconectarse (usabilidad). Ninguno de los dos valores puede sacrificarse completamente.

Arquitectura de autenticación moderna

JWT stateless es ligero y escalable pero sin control de revocación. Las sesiones tradicionales en BD son controlables pero crean estado en el servidor. El enfoque híbrido (JWT + tabla de sesiones) combina lo mejor de ambos mundos: los tokens son verificables sin BD en el caso normal, pero la BD permite revocar cuando es necesario.

Comunicación entre pestañas con BroadcastChannel

La BroadcastChannel API permite que múltiples contextos del mismo origen (pestañas, iframes, workers) se comuniquen en tiempo real sin necesidad de un servidor WebSocket. Es la solución nativa para sincronizar estado entre pestañas y eliminar inconsistencias de UI sin añadir complejidad de infraestructura.

Tests como documentación viva

Los 14 tests escritos no solo validan el comportamiento, también documentan los escenarios que el sistema debe manejar. Un desarrollador nuevo puede leer los nombres de los tests y entender exactamente qué garantiza el sistema de sesiones.
